

25006. [Pro] 전기차충전소

문제 내용



- 시간 : 25개 테스트케이스를 합쳐서 C++의 경우 3초 / Java의 경우 3초
- 메모리 : 힙, 정적 메모리 합쳐서 256MB 이내, 스택 메모리 1MB 이내

※ 문제 및 풀이에 대해 **온라인/오프라인에 게시/공유 하는 것은 금지**되어 있습니다.

[제한 사항]

시간	25개 테스트케이스를 합쳐서 C++ 의 경우 3초 / Java 의 경우 3초
메모리	힙, 정적 메모리 합쳐서 256MB 이내, 스택 메모리 1MB 이내

- ① C 또는 C++로 답안을 작성하시는 응시자께서는 검정시스템에 제출 시, **Language** 에서 C++ 를 선택하신 후 제출하시기 바랍니다.
- ② **Main** 과 **User Code** 부분으로 구성되어 있습니다.
 - A. Main : 수정할 수 없는 코드이며, 채점 시 비 정상적인 답안 검출 등 평가를 위한 로직이 추가 될 수 있습니다.
 - B. User Code : 실제 응시자가 작성해야 하는 코드이며, 제출 시에는 표준 입출력 함수가 포함되어 있으면 안 됩니다.
- ③ **Local PC** 에서 프로그래밍 시 유의 사항
 - A. 2개의 파일을 생성하셔야 합니다. (main.cpp / solution.cpp 또는 Solution.java / UserSolution.java)
 - B. Main 부분의 코드를 main.cpp 또는 Solution.java 에 복사해서 사용하시기 바랍니다.
 - C. sample_input.txt 를 사용하시기 위해서는 Main 부분의 코드 내에
표준 입력을 파일로 전환하는 코드 (주석처리 되어 있음) 의 주석을 풀어서 사용하시면 됩니다.
 - D. User Code 부분의 코드를 작성하신 후 서버에 제출하실 때,
디버깅을 위한 표준 입출력 함수를 모두 삭제 또는 주석 처리해 주셔야 합니다.
- ④ 문제 내에 제약조건을 모두 명시하지 않으므로 주어지는 코드를 분석하셔야 합니다.
- ⑤ 코드는 개발 언어에 따라 상이할 수 있으므로, 작성할 언어를 기준으로 분석하셔야 합니다.

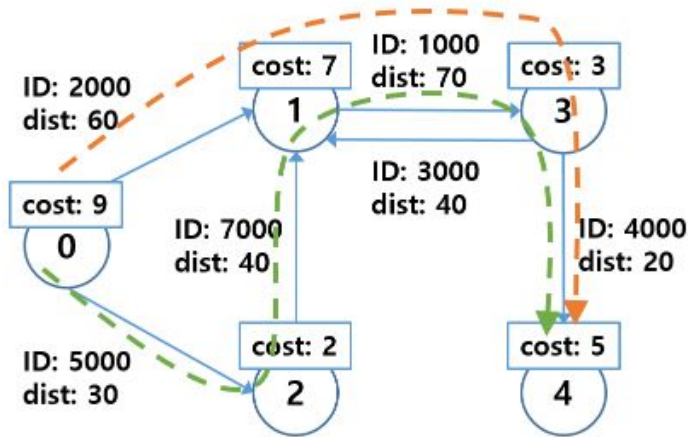
[문제 설명]

N개의 도시가 주어진다. 각 도시는 0부터 N-1까지 ID값을 가진다. 그리고 각 도시마다 1개의 전기차 충전소가 존재한다. 각 충전소는 단위 거리당 충전 비용을 가지고, 전기자의 충전 용량은 무한대이다.

도시를 연결하는 단방향 도로가 추가될 때, 해당 도로의 ID와 거리가 주어진다.

출발 도시에서 도착 도시로 이동할 때 필요한 최소 충전 비용을 구하고자 한다.
전기차의 충전 용량은무한대이기 때문에 충전 비용이 낮은 충전소에서 충전을 미리 하는 것이 유리하다.

[Fig. 1]은 5개의 도시가 주어진 예이다.



[Fig. 1]

(0 --> 1 --> 3 --> 4) 경로로 가는 경우, 0번 도시에서 $9 \times 60 = 540$ 비용으로 충전하고, 1번 도시로 이동하여 $7 \times 70 = 490$ 비용으로 충전하고, 3번 도시로 이동하여 $3 \times 20 = 60$ 비용으로 충전하면, 총 1090 비용으로 4번 도시에 도착할 수 있다.

(0 --> 2 --> 1 --> 3 --> 4) 경로로 가는 경우, 0번 도시에서 $9 \times 30 = 270$ 비용으로 충전하고, 2번 도시로 이동한다. 2번 도시의 충전 비용이 1, 3, 4번 도시보다 낮기 때문에 여기서 4번까지 가기 위해 필요한 모든 전기를 충전한다. 그 비용은 $2 \times (40 + 70 + 20) = 260$ 이다. 따라서 총 530 비용으로 4번 도시에 도착할 수 있고, 이 비용이 0번 도시에서 4번 도시로 가기 위해 필요한 최소 충전 비용이다.

아래 API 설명을 참조하여 각 함수를 구현하라.

※ 아래 함수 signature는 C/C++에 대한 것으로 다른 언어에 대해서는 제공되는 Main과 User Code를 참고하라.

아래는 User Code 부분에 작성해야 하는 API 의 설명이다.

void init(int N, int mCost[], int K, int mld[], int sCity[], int eCity[], int mDistance[])
각 테스트 케이스의 처음에 호출된다.
N개의 도시가 주어진다. 각 도시는 0부터 N-1까지 ID값을 가진다.
N개의 단위 거리당 충전 비용이 mCost 배열로 주어진다.
K개의 도로 정보가 주어진다. 각 도로의 ID, 출발 도시와 도착 도시, 그리고 거리가 주어진다.
단방향 도로이기 때문에 출발 도시에서 도착 도시로만 갈 수 있다.
출발 도시와 도착 도시의 순서쌍이 동일한 도로는 없다.
출발 도시와 도착 도시가 서로 같은 경우는 없다.
도로 ID가 서로 같은 경우는 없다.
Parameters
N: 도시의 개수 ($5 \leq N \leq 300$)
($0 \leq i < N$)인 모든 i에 대해,

mCost[i]: 도시 i의 단위 거리당 충전 비용 ($1 \leq \text{mCost}[i] \leq 2,000$)

K: 도로의 개수 ($4 \leq K \leq 2,000$)

$(0 \leq i < K)$ 인 모든 i 에 대해,

mld[i]: 도로 i의 ID ($1 \leq \text{mld}[i] \leq 1,000,000,000$)

sCity[i]: 도로 i의 출발 도시 ($0 \leq \text{sCity}[i] < N$)

eCity[i]: 도로 i의 도착 도시 ($0 \leq \text{eCity}[i] < N$)

mDistance[i]: 도로 i의 거리 ($1 \leq \text{mDistance}[i] \leq 2,000$)

```
void add(int mld, int sCity, int eCity, int mDistance)
```

출발 도시가 sCity이고, 도착 도시가 eCity이고, 거리가 mDistance인 mld 도로를 추가한다.

단방향 도로이기 때문에 출발 도시에서 도착 도시로만 갈 수 있다.

init0에 없던 새로운 도시는 주어지지 않는다.

sCity에서 eCity로 가는 도로가 이미 존재하는 경우는 없다.

sCity와 eCity가 서로 같은 경우는 없다.

mld 값으로 이미 존재하는 도로의 ID가 주어지는 경우는 없다.

Parameters

mld: 도로의 ID ($1 \leq mld \leq 1,000,000,000$)

sCity: 도로의 출발 도시 ($0 \leq \text{sCity} < N$)

eCity: 도로의 도착 도시 ($0 \leq \text{eCity} < N$)

mDistance: 도로의 거리 ($1 \leq mDistance \leq 2,000$)

```
void remove(int mld)
```

mld 도로를 제거한다.

존재하지 않는 도로의 ID가 주어지는 경우는 없다.

Parameters

mld: 도로의 ID ($1 \leq mld \leq 1,000,000,000$)

```
int cost(int sCity, int eCity)
```

sCity에서 eCity로 가는데 필요한 최소 충전 비용을 반환한다.

sCity와 eCity가 서로 같은 경우는 없다.

Parameters

sCity: 출발 도시 ($0 \leq \text{sCity} < N$)

eCity: 도착 도시 ($0 \leq \text{eCity} < N$)

Returns

sCity에서 eCity로 이동이 가능하다면, 필요한 최소 충전 비용을 반환한다.

불가능하다면, -1을 반환한다.

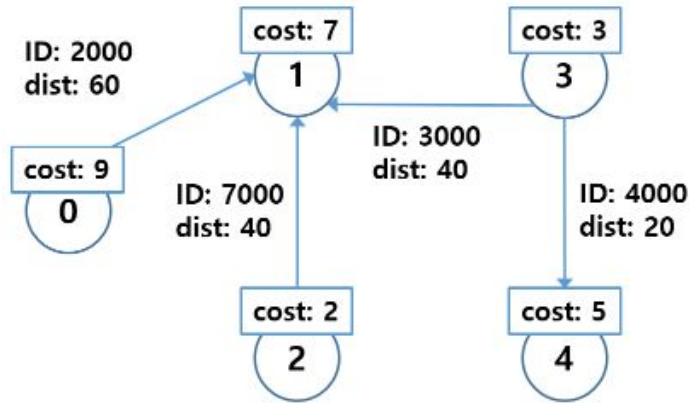
[예제]

아래의 [Table 1]과 같이 요청이 되는 경우를 살펴보자.

Order	Function	return	Figure
1	init(5, {9,7,2,3,5}, 4, {2000,7000,4000,3000}, {0,2,3,3}, {1,1,4,1}, {60,40,20,40})		Fig. 2
2	cost(1, 4)	-1	
3	add(5000, 0, 2, 30)		
4	add(1000, 1, 3, 70)		Fig. 3
5	cost(0, 4)	530	Fig. 4
6	remove(5000)		Fig. 5
7	cost(0, 4)	1090	Fig. 6

[Table 1]

(순서 1) 5개 도시의 단위 거리당 충전 비용과 4개의 도로 정보가 주어진다. 함수 호출의 결과는 [Fig. 2]와 같다.

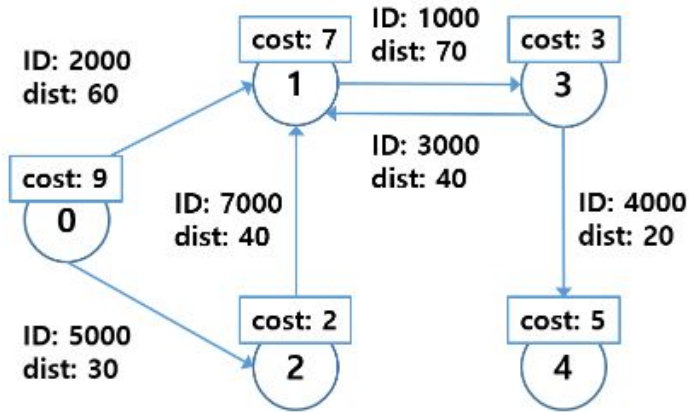


[Fig. 2]

(순서 2) 1번 도시에서 4번 도시로 가는데 필요한 최소 충전 비용을 계산한다. 이동할 수 있는 방법이 없기 때문에 -1을 반환한다.

(순서 3) 0번 도시에서 2번 도시로 갈 수 있는 5000 도로가 추가된다. 도로의 거리는 30이다.

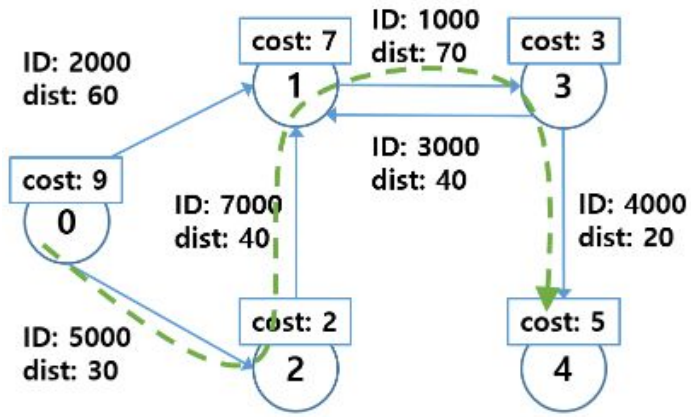
(순서 4) 1번 도시에서 3번 도시로 갈 수 있는 1000 도로가 추가된다. 도로의 거리는 70이다. 함수 호출의 결과는 [Fig. 3]과 같다.



[Fig. 3]

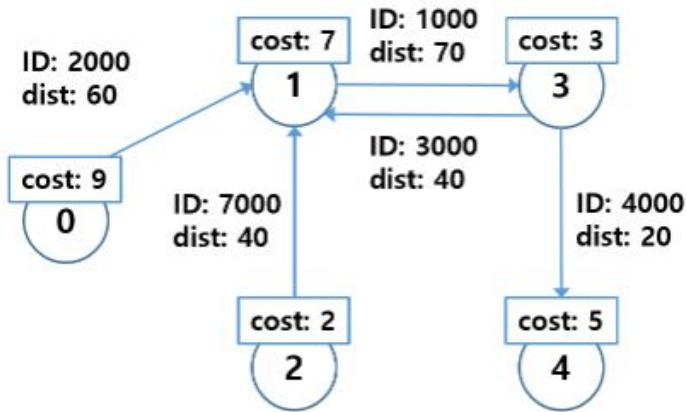
(순서 5) 0번 도시에서 4번 도시로 가는데 필요한 최소 충전 비용을 계산한다. [Fig. 4]와 같이 최소 충전 비용으로 이동할 수 있기 때문에, 530을 반환한다.

(0번 도시에서 $9 \times 30 = 270$ 비용으로 충전하고, 2번 도시로 이동한다.
2번 도시의 충전 비용이 1, 3, 4번 도시보다 낮기 때문에 여기서 4번까지 가기 위해 필요한 모든 전기를 충전한다.
그 비용은 $2 \times (40 + 70 + 20) = 260$ 이다. 따라서 총 530 비용으로 4번 도시에 도착할 수 있다.)



[Fig. 4]

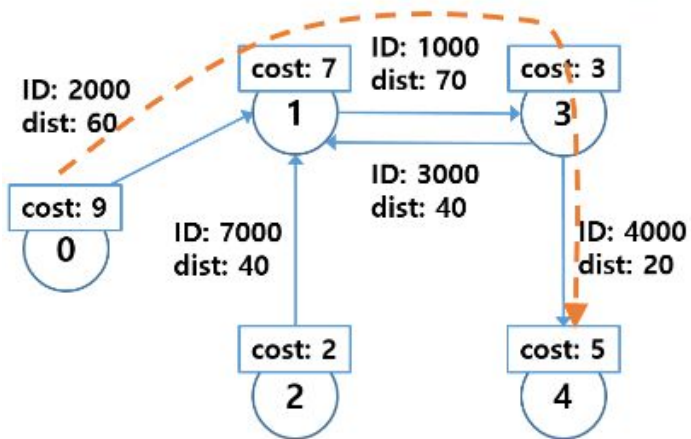
(순서 6) 5000 도로를 제거한다. 함수 호출의 결과는 [Fig. 5]와 같다.



[Fig. 5]

(순서 7) 0번 도시에서 4번 도시로 가는데 필요한 최소 충전 비용을 계산한다. [Fig. 6]과 같이 최소 충전 비용으로 이동할 수 있기 때문에, 1090을 반환한다.

(0번 도시에서 $9 \times 60 = 540$ 비용으로 충전하고, 1번 도시로 이동하여 $7 \times 70 = 490$ 비용으로 충전하고, 3번 도시로 이동하여 $3 \times 20 = 60$ 비용으로 충전하면, 총 1090 비용으로 4번 도시에 도착할 수 있다.)



[Fig. 6]

[제약사항]

1. 각 테스트 케이스 시작 시 `init()` 함수가 호출된다.
2. 각 테스트 케이스에서 `add()` 함수의 호출 횟수는 1,400 이하이다.
3. 각 테스트 케이스에서 `remove()` 함수의 호출 횟수는 500 이하이다.
4. 각 테스트 케이스에서 `cost()` 함수의 호출 횟수는 100 이하이다.

[입출력]

입출력은 제공되는 Main 부분의 코드에서 처리하므로 User Code 부분의 코드에서는 별도로 입출력을 처리하지 않는다.

Sample input 에 대한 정답 출력 결과는 "#TC번호 결과" 의 포맷으로 보여지며 결과가 100 일 경우 정답, 0 일 경우 오답을 의미한다.